

Vue.js 마무리 강의

Vite 프로젝트 빌드 & Node.js 배포 실습



STEP 1

Vite Build



STEP 2

Express Server



STEP 3

Deploy

오늘 수업의 목표

“

지금까지 우리는 브라우저에서 `npm run dev`로 개발 서버만 띄워서 작업했습니다.
오늘은 이 Vue 프로젝트가 실제 서비스로 배포되는 실습을 진행합니다.



STEP 01

프로젝트 빌드

Vite를 사용하여 프로덕션용 빌드 명령
실행



STEP 02

결과물 이해

dist 폴더의 구조와 생성된 파일들의
역할 파악



STEP 03

서버 실행

Node.js Express 위에서 실제
서비스처럼 구동



STEP 04

체크 포인트

배포 시 실수하기 쉬운 필수 확인 사항
점검

개발 서버 vs 빌드 결과물



개발 중 (Development)

연습 및 구현 단계



\$ npm run dev

⚡ Vite 개발 서버 & Hot Reload

코드 수정 시 브라우저에 즉시 반영되어 빠른 개발 가능

👁 소스코드 노출

디버깅을 위해 소스맵(Source Map)이 포함되어 코드가 그대로 보임

🚫 실제 서비스용 아님 ❌

최적화가 되어있지 않아 파일 크기가 크고 느릴 수 있음

VS



배포용 (Production)

실제 사용자 제공 단계



\$ npm run build

📄 최적화된 결과물 생성

불필요한 공백 제거, 코드 압축(Minify), 난독화(Uglify) 적용

📦 정적 파일 번들링

HTML, CSS, JS 이미지가 최적화되어 dist 폴더에 생성됨

✅ 실제 서비스에 적합 ✔️

로딩 속도가 빠르고 보안성이 강화된 상태



"dev는 개발 연습용, build는 실전 배포용"이라고 생각하면 됩니다.

Vite 빌드 명령 실행

`npm run build`

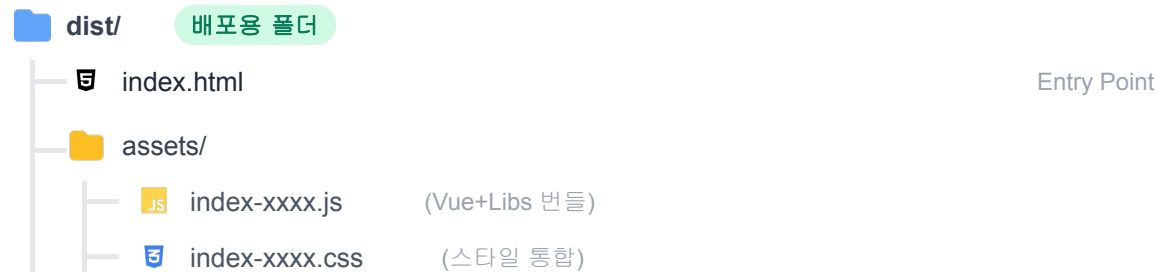
`.env.local` → `.env.production`

빌드 프로세스



생성 결과물 구조

`./dist`



핵심 포인트

- ✗ **src/ 폴더는 배포하지 않음**
소스 코드는 브라우저가 이해할 수 있는 형태로 변환되었으므로 원본은 불필요
- ✓ **dist/ 폴더만 있으면 서비스 가능**
Vue, Vuetify, Router 등 모든 의존성이 하나로 합쳐져 포함됨
- 📦 **HTML + JS + CSS 묶음 파일**
이제 Vue 프로젝트는 복잡한 프레임워크가 아닌 단순한 정적 파일들의 모음이 됨

로컬에서 빌드 결과 미리보기

`npm run preview`

서버 동작 방식 차이

Dev
Server



JS

소스코드 실시간 컴파일

Preview



빌드된 파일(dist) 제공

🔍 왜 Preview가 필수인가요?

1

실제 배포 환경 시뮬레이션

개발 서버(dev)는 HMR 등 개발 편의 기능을 제공하지만, 실제 브라우저 동작과 미묘하게 다를 수 있습니다. Preview는 빌드된 결과물을 그대로 실행합니다.

2

숨겨진 오류 발견

"개발할 땐 잘 됐는데 배포하니 안 돼요"의 주범인 경로 문제, 이미지 누락, 환경 변수 미적용 등을 미리 찾을 수 있습니다.

📋 배포 전 필수 체크리스트

새로고침 시 페이지가 정상적으로 뜨는가? (404 에러 체크)

로고나 배경 이미지가 깨지지 않고 잘 나오는가?

API 요청이 실제 서버 주소로 잘 나가는가?

“ dev는 연습, preview는 리허설, build는 공연입니다 ”

Node.js로 배포 서버 만들기

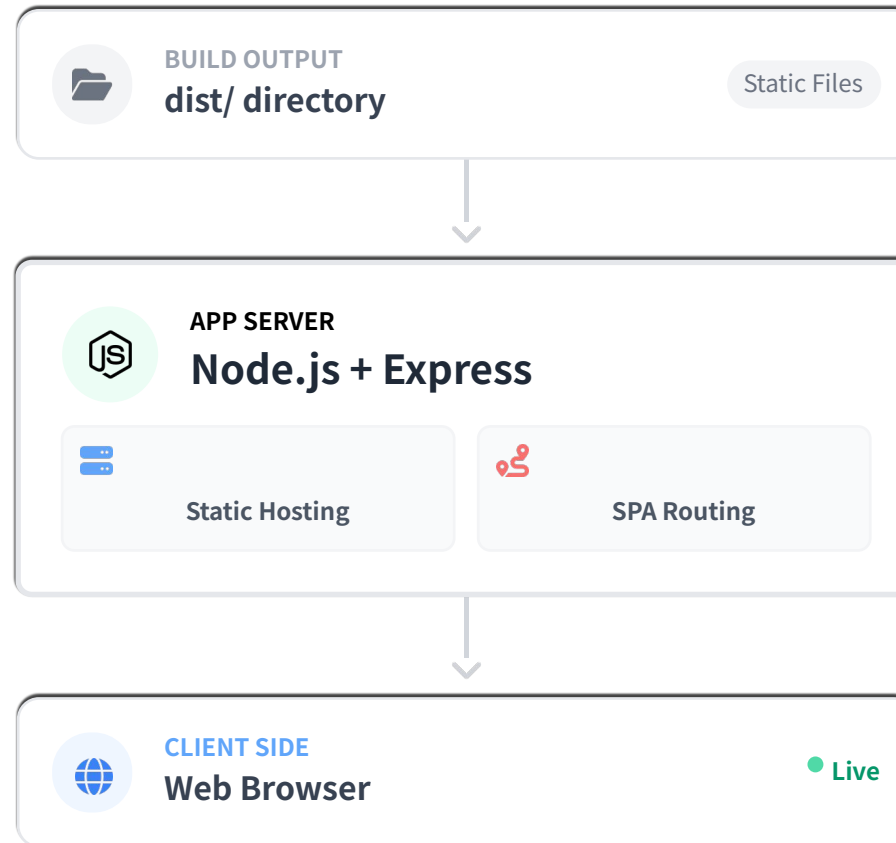
Express 패키지 설치

```
npm init -y  
npm install express
```

```
project-root/  
├─ dist/  
│   ├─ index.html  
│   └─ assets/  
├─ app.js  
└─ package.json
```

Server Architecture Flow

i Express는 Node.js를 위한 빠르고 개방적인 웹 프레임워크입니다. 정적 파일 호스팅과 라우팅 처리를 위해 사용합니다.



서버 파일 생성 (server.js)

```
const express = require('express')
const path = require('path')

const app = express()
const DIST_DIR = path.join(__dirname, 'dist')

// 1) dist 폴더를 정적 파일로 서비스
app.use(express.static(DIST_DIR))

// 2) SPA fallback (Vue Router history mode)
//   어떤 경로로 들어와도 dist/index.html 반환
app.get(/.*/, (req, res) => {
  res.sendFile(path.join(DIST_DIR, 'index.html'))
})

const PORT = process.env.PORT || 3000
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`)
})
```



정적 파일 서비스

```
express.static('dist')
```

빌드된 dist/ 폴더 내의 HTML, CSS, JS, 이미지 파일들을 브라우저가 직접 접근할 수 있도록 공개합니다.



SPA 새로고침 문제 해결

```
app.get('*', ...)
```

Vue Router가 사용하는 URL 경로(예: /login, /board)는 실제 파일이 아닙니다. 서버가 404 에러 대신 무조건 index.html을 돌려주도록 설정하여 프론트엔드가 라우팅을 처리하게 합니다.

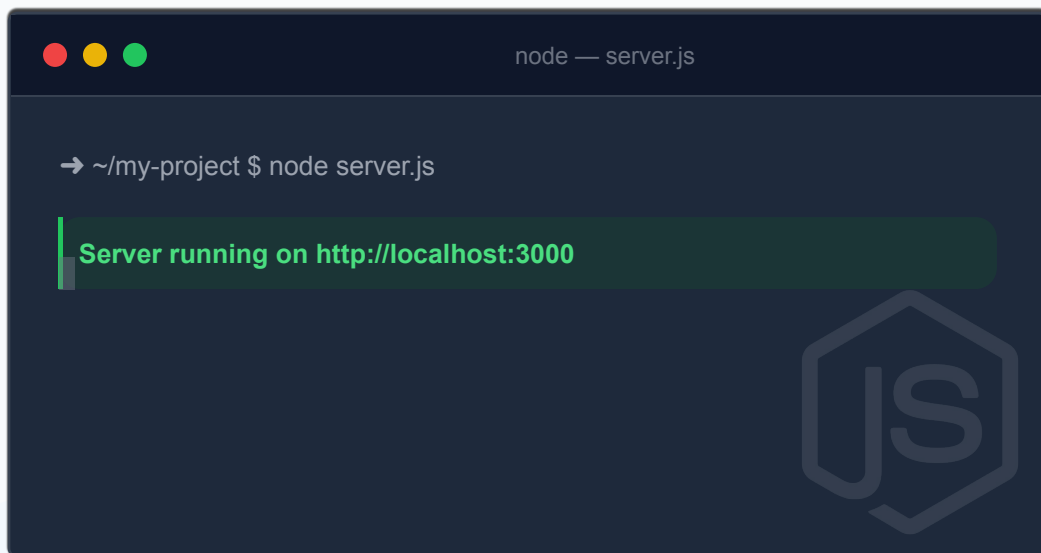


실무 배포 구조의 핵심

- ✓ 이 코드 하나로 실제 웹 서비스와 동일한 구조가 완성됩니다.
- ✓ 개발 서버(Vite)는 더 이상 필요 없으며, Node.js가 빌드된 파일만 전달합니다.

서버 실행 및 접속 확인

2 서버 실행 (Start Server)

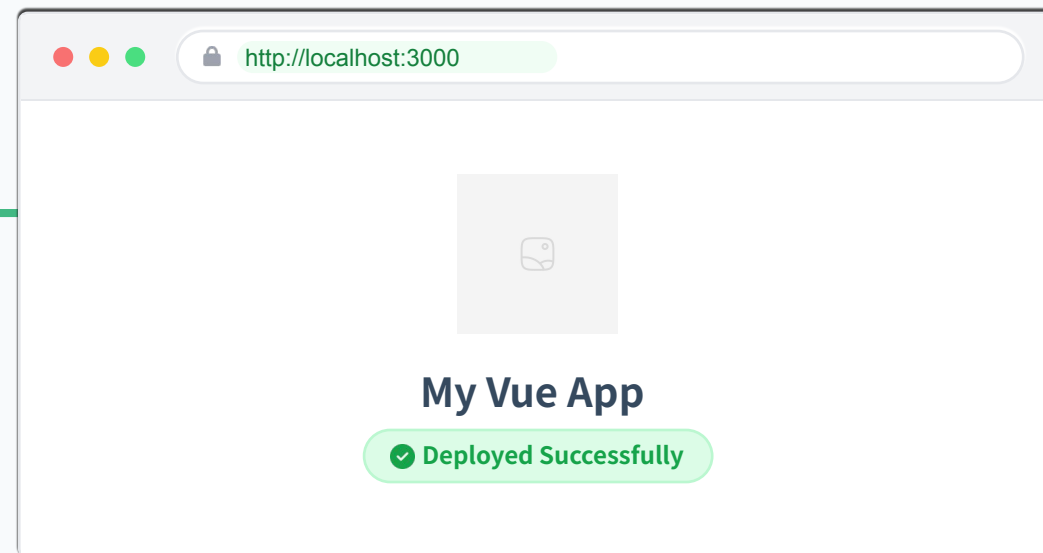
A terminal window titled 'node — server.js' with a dark background. It shows the command '→ ~/my-project \$ node server.js' and a green message 'Server running on http://localhost:3000'. A large, faint 'JS' logo is visible in the background.

```
node — server.js

→ ~/my-project $ node server.js

Server running on http://localhost:3000
```

3 브라우저 접속 (Access)



지금 여러분의 컴퓨터(Localhost)가 서버 역할을 하고, 브라우저가 클라이언트 역할을 하고 있습니다. AWS EC2나 Docker 환경에 배포할 때도 이 'dist 폴더 + Node 서버' 구조를 그대로 가져갑니다.

필수 포인트: 환경 변수 (Vite)

.env

Project Root

```
1 VITE_API_URL https://api.example.com
2 DB_PASSWORD secret123
```

❗ 클라이언트 노출 안됨



src/api/index.js

```
1 // ✅ VITE_ 접두사가 있어야 접근 가능
2 const apiUrl Import.meta.env.VITE_API_URL
3 // ❌ 접두사가 없으면 undefined
4 const pw Import.meta.env.DB_PASSWORD
```

🔑 Vite 환경 변수 규칙

1

VITE_ 접두사 필수

보안상의 이유로 VITE_로 시작하지 않는 변수는 클라이언트 번들에 포함되지 않습니다.

2

import.meta.env 객체 사용

Node.js의 process.env 대신 Vite 전용 객체를 통해 접근해야 합니다.

⚠ 주의: 빌드 타임 주입 (Build-time Injection)

환경 변수는 런타임에 읽어오는 것이 아니라, 빌드 시점에 문자열로 대체 (Replacement)됩니다.

// 빌드 전 코드

import.meta.env.VITE_API



// 빌드 후 코드 (실제 값으로 박제됨)

"https://api.example.com"

따라서 .env 값을 바꾸면 반드시 재빌드(npm run build)해야 적용됩니다.

API 키 노출과 보안 전략

현실 (Reality)

Developer Tools - Sources

```
const firebaseConfig = {  
  apiKey: "AIzaSyB2...",  
  authDomain: "...",  
  projectId: "..."  
};  
// 누구나 볼 수 있음!
```

"프론트엔드 키는 숨길 수 없다"

브라우저가 서버와 통신하려면 키가 필요하기 때문에, 소스코드(JS 번들) 안에 키가 포함되어 배포됩니다. 개발자 도구만 열어도 누구나 확인할 수 있습니다.



대응 전략 (How to Secure)



1. 인증/인가 규칙 적용 (Rules)

키가 노출되어도 로그인한 사용자만, 자신의 데이터만 접근하도록 데이터베이스 규칙 (Firestore Rules)을 설정합니다.



2. 도메인 제한 (Domain Restriction)

Google Cloud Console 등에서 API 키가 특정 도메인(예: mysite.com)에서만 호출되도록 제한을 겁니다.



3. 민감한 로직은 백엔드로 이동

결제 비밀번호, 관리자 권한 키(Service Account) 등 절대 노출되면 안 되는 정보는 Node.js 서버나 Cloud Functions에서 처리합니다.



절대 금지: Service Account Key

서버용 관리자 키(private_key 등)는 절대 프론트엔드 코드나 .env에 포함시키지 마세요! 이는 서버의 모든 권한을 넘겨주는 것과 같습니다.

배포 시 절대 올리지 않는 것

❌ 절대 업로드 금지 (Don't)

이 파일들은 소스코드이거나 보안 정보입니다.
서버 실행에는 필요하지 않습니다.

**node_modules/**

용량이 너무 크고, 서버에서 npm install로 설치해야 함

**src/**

원본 소스코드(Vue 파일 등)는 브라우저가 해석 못 함

**.env**

API 키, DB 접속 정보 등 민감한 비밀 정보 포함



✅ 배포할 항목 (Do)

서버는 오직 실행 결과물만 있으면 충분합니다.
가볍고 빠르게 배포하세요.

**dist/**

빌드 완료된 정적 파일 모음

index.html, assets(js, css, img)



배포 원칙 (Deployment Principle)

"소스코드(src)를 올리는 게 아니라, 결과물(dist)을 올리는 것입니다."

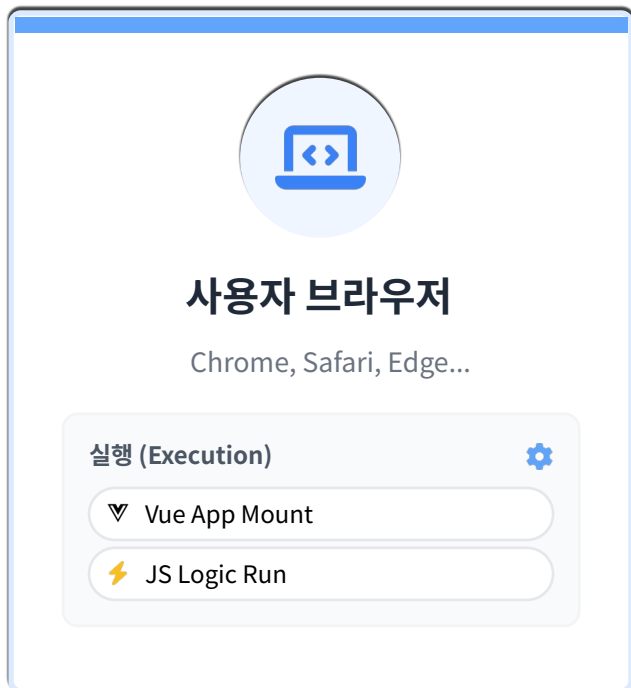
**server.js**

파일을 서빙할 최소한의 서버 코드

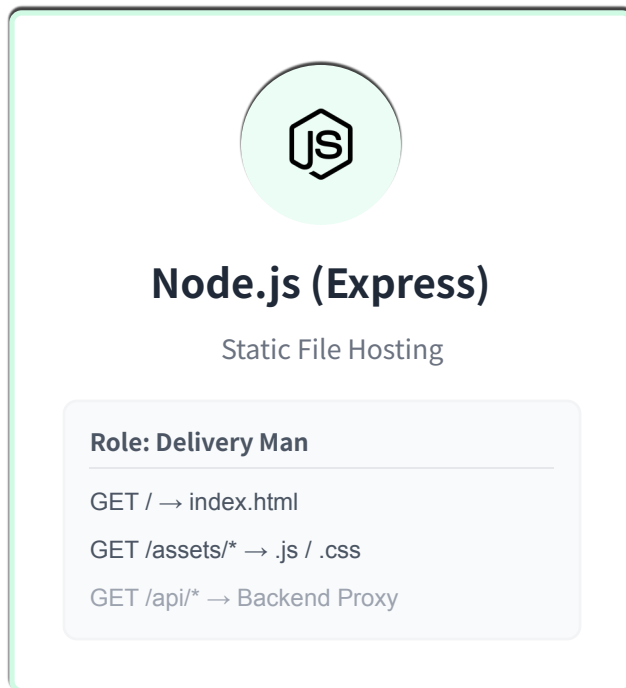


실제 서비스 구조 요약

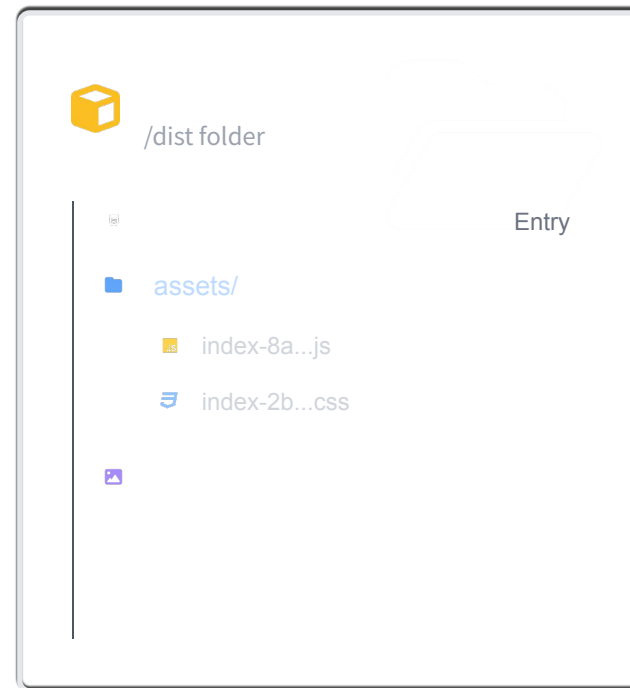
CLIENT SIDE



WEB SERVER



DISK STORAGE



핵심 요약 (Key Takeaway)

Vue 앱은 서버에서 실행되는 것이 아닙니다. 서버는 단순히 파일을 전달(Serving)하고, 실제 앱 구동은 사용자의 브라우저에서 일어납니다. 이것이 CSR(Client Side Rendering)의 기본 구조입니다.

COURSE COMPLETED

Vue.js 마스터 클래스 완강

“

여러분은 이제 Vue 컴포넌트 개발부터 배포까지 전 과정을 경험했습니다.

“이 흐름을 이해한 사람은 ‘화면을 만드는 사람’이 아니라
‘서비스를 만드는 개발자’입니다.”

”



Component

컴포넌트 설계 및 개발



State Mgmt

Pinia 상태 관리



API Integration

서버 통신 및 데이터 처리



Deploy

Vite 빌드 & Node.js 배포



Firestore 설정과 .env의 관계

💡 .env 파일은 저장소에 올리지 않지만, Firestore 설정값은 빌드 결과물(프론트엔드)에 반드시 포함됩니다.
(이 둘은 모순이 아닙니다!)

“가장 쉬운 비유 (Analogy)”



.env 파일 = 비법 레시피 노트

주방장(개발자)만 봐야 함. 손님(GitHub)에게 보여주지 않음



빌드 결과물 (dist) = 완성된 요리

손님(브라우저)에게 서빙됨. 재료(API Key)가 이미 섞여서 포함됨

🛡️ 보안의 진실 (Security Reality)

"그럼 키가 노출되면 위험한 거 아닌가요?"

Firestore의 공식 입장: "API Key는 비밀번호가 아니라, 프로젝트를 찾는 식별자(Identifier)입니다."

🔒 진짜 보안을 담당하는 곳:

보안 대상	담당 기능 (Where)
사용자 인증	Firebase Auth
데이터 접근 권한	Firestore Rules (가장 중요!)
관리자/삭제 권한	Backend Server Only

“보안은 숨기는 게 아니라, 검증(Verification)하는 것입니다.”